CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: Pawfolio

Prepared By: Arong, Kylene Jacaban

Version: 1.0

Date: 18/02/2026
Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	18/02/2026	Kylene J. Arong	Initial draft	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 9

4.0 SYSTEM ARCHITECTURE 11

5.0 API CONTRACT & COMMUNICATION 12

6.0 DATABASE DESIGN 17

7.0 UI/UX DESIGN 18

8.0 PLAN 21

EXECUTIVE SUMMARY

1.1 Project Overview

Pawfolio is a pet health record management application that allows pet owners to securely store, manage, and track their pets' medical information. The system consists of a Spring Boot backend API, a React web application, and an Android mobile application, all integrated to provide a consistent and accessible experience across platforms.

1.2 Objectives

1. Develop a pet health record management system with secure user authentication.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Create RESTful APIs for managing pet profiles and health records
4. Design a user-friendly interface for viewing and updating pet medical history.
5. Ensure data security and reliability for sensitive pet health information

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- User profile management
- Pet profile creation and management
- Pet health record management (vaccinations, checkups, medications)
- Dashboard overview of pets and recent records
- Responsive web interface
- Native Android mobile application
- Relational database using MySQL or PostgreSQL

Excluded Features:

- Online veterinary appointment booking
- Push notifications or reminders
- Integration with external veterinary systems

- File uploads for medical documents
- Social media login integration
- Advanced analytics or reporting

1.0 INTRODUCTION

1.1 Purpose

This document provides the comprehensive Software Design Description (SDD) for the Pawfolio system. It defines the system architecture, functional and non-functional requirements, API design, and implementation scope to ensure seamless integration between backend, web, and mobile components.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Pawfolio
Domain: Pet Care / Health Management
Primary Users: Pet Owners

Problem Statement: Pet owners often struggle to keep track of their pets' medical records across clinics and physical documents.
Solution: Pawfolio provides a centralized digital platform for storing and managing pet health records, accessible anytime through web or mobile applications.

2.2 Core User Journeys

Journey 1: First-time User Setup

1. User opens the web or mobile application
2. Registers an account using email and password
3. Logs into the system
4. Navigates to the dashboard
5. Creates a pet profile
6. Adds initial health records for the pet

Journey 2: Managing Pet Health Records

1. User logs in with existing credentials
2. Selects a pet from the dashboard

3. Views the pet’s health history
4. Adds a new health record (e.g., vaccination or checkup)
5. Updates or deletes existing records

Journey 3: Profile Management

1. User logs in
2. Navigates to profile page
3. Updates personal information
4. Logs out securely

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. User profile management
3. Pet profile management (add, edit, delete pets)
4. Pet health record management (CRUD operations)
5. Dashboard overview

SHOULD HAVE

1. Categorization of health records (vaccination, medication, checkup)
2. Search or filter health records by date or type
3. Input validation and error handling
4. Responsive UI design

COULD HAVE

1. Health record summaries per pet
2. Basic reminder notes (non-notification based)

WON'T HAVE

1. Vet appointment scheduling
2. Push notifications
3. Third-party API integrations

4. File attachments for records
5. Social login

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** JWT authentication, bcrypt password hashing

Feature: Pet Profile Management

- **Screens:** Dashboard, Add/Edit Pet
- **Fields:** Pet name, species, breed, age
- **Functions:** Add, edit, delete pet profiles
- **API Endpoints:** GET /pets, POST /pets, PUT /pets, DELETE /pets/{id}
- **Admin Functions:** POST /products, PUT /products/{id}, DELETE /products/{id}

Feature: Pet Health Records

- **Screens:** Pet Detail, Add/Edit Health Record
- **Record Types:** Vaccination, medication, checkup
- **Fields:** Record type, description, date, notes
- **Functions:** Add, view, update, delete health records
- **Persistence:** Records stored per pet per user
- **API Endpoints:** GET /pets/{petId}/records, POST /pets/{petId}/records, PUT /records/{id}, DELETE /records/{id}

2.5 Acceptance Criteria

AC-1: Successful User Registration

Given I am a new user

When I enter a valid email and password

And confirm my password

Then my account should be created

And I should be logged in

And redirected to the dashboard

AC-2: Add Pet Health Record

Given I am logged in

And I have an existing pet profile

When I add a new health record with valid details

Then the record should be saved

And displayed in the pet's health history

AC-3: View Pet Records

Given I am logged in

When I select a pet

Then I should see all health records associated with that pet

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 2 seconds for 95% of requests
- Dashboard load time ≤ 3 seconds
- Mobile app cold start ≤ 3 seconds
- Support at least 100 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- HTTPS for all communications
- JWT-based authentication

- Password hashing using bcrypt (salt rounds = 12)
- SQL injection prevention
- XSS protection
- Role-based access to user-owned data

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile, tablet, desktop
- **Operating Systems:** Windows, macOS, Linux

3.4 Usability Requirements

- New users can add a pet within 5 minutes
- Simple and consistent navigation
- Clear error and validation messages
- Mobile-friendly touch targets
- Accessible UI components

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram

```
┌─────────────────┐  ┌─────────────────┐
│  WEB CLIENT   │  │  MOBILE CLIENT │
```

```
│   (React)     │  │  (Android)    │
│  • TypeScript │  │  • Kotlin     │
│  • Tailwind CSS│  │  • Compose    │
└───────┬───────┘  └───────┬───────┘
        │                  │
        └─────────┬────────┘
                  │
        ┌─────────▼─────────┐
        │  BACKEND API      │
        │  (Spring Boot)    │
        │  • Java 17        │
        │  • Spring Security│
        │  • JPA/Hibernate  │
        └─────────┬─────────┘
                  │
        ┌─────────▼─────────┐
        │  DATABASE         │
        │  (PostgreSQL)     │
        │  • Users table    │
        │  • Products table │
        │  • Orders table   │
        │  • Cart items table│
        └───────────────────┘
```

Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios

- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** https://api.marketplace.com/api/v1
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **Response Structure:**

```json
{
  "success": boolean,
  "data": object|null,
  "error": {
    "code": string,
    "message": string,
    "details": object|null
  },
  "timestamp": string
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST   /auth/register

Body: {email, password, confirmPassword, fullName?}

Response: {user: {id, email, name}, token, refreshToken}

POST   /auth/login

Body: {email, password}

Response: {user: {id, email, name, role}, token, refreshToken}

POST   /auth/logout

Headers: Authorization: Bearer {token}

Response: {message: "Logged out successfully"}

Product Endpoints

GET   /products

Query: ?page=1&limit=20&search=keyword&category=electronics

Response: {products: [...], pagination: {page, limit, total, pages}}

GET   /products/{id}

Response: {product: {id, name, description, price, stock, imageUrl, category}}

POST   /products (Admin only)

Body: {name, description, price, stock, imageUrl, category}

Response: {product: {...}}

PUT   /products/{id} (Admin only)

Body: {name?, description?, price?, stock?, imageUrl?, category?}

Response: {product: {...}}

Cart Endpoints

GET   /cart (Authenticated)

Response: {cart: {id, items: [...], total, itemCount}}

POST   /cart/items

Body: {productId, quantity}

Response: {message: "Added to cart", cartItem: {...}}

PUT   /cart/items/{itemId}

Body: {quantity}

Response: {message: "Cart updated", cartItem: {...}}

DELETE /cart/items/{itemId}

Response: {message: "Removed from cart"}

Order Endpoints

POST   /orders

Body: {shippingAddress: {fullName, address, city, zipCode, country}}

Response: {order: {id, orderNumber, total, status, items: [...], createdAt}}

GET   /orders

Response: {orders: [...]}

GET   /orders/{id}

Response: {order: {...}}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created

- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

json

```json
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2024-01-28T10:30:00Z"
}

{
  "success": false,
  "data": null,
  "error": {
    "code": "VALID-001",
    "message": "Validation failed",
    "details": {
      "email": "Email is required",
```

```
      "password": "Must be at least 8 characters"
    }
  },
  "timestamp": "2024-01-28T10:30:00Z"
}
```

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD

```
USERS (1) ——— (1) CARTS (1) ——— (*) CART_ITEMS (*) ——— (1) PRODUCTS
   |                  |
   |                  |
  (*)                (*)
   |                  |
   |                  |
ORDERS (1) ——— (*) ORDER_ITEMS (*) ———┘
```

Detailed Relationships:

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)

- **One-to-Many:** User → Orders (User can have multiple orders)
- **One-to-Many:** Cart → CartItems (Cart contains multiple items)
- **One-to-Many:** Order → OrderItems (Order contains multiple items)
- **Many-to-One:** CartItems → Product (Items reference products)
- **Many-to-One:** OrderItems → Product (Items reference products)

Key Tables:

1. **users** - User accounts and authentication
2. **products** - Product catalog information
3. **carts** - Shopping cart per user
4. **cart_items** - Items in shopping cart
5. **orders** - Customer orders
6. **order_items** - Items in each order
7. **refresh_tokens** - JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **products:** id, name, description, price, stock, image_url, category
- **carts:** id, user_id, created_at
- **cart_items:** id, cart_id, product_id, quantity
- **orders:** id, order_number, user_id, total, status, shipping_address
- **order_items:** id, order_id, product_id, product_name, quantity, price

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Homepage (Product Listing)

Header: [Logo] [Search Bar] [Cart Icon] [User Menu]

Content: Product Grid (3 columns desktop)

Each Product Card: Image, Name, Price, "Add to Cart" button

Footer: Links, Copyright

Product Detail Page

Back Button

Product Image (large)

Product Name and Price

Description

Quantity Selector (1-10)

"Add to Cart" and "Buy Now" buttons

Product Specifications

Shopping Cart Page

Cart Title

List of Cart Items (Image, Name, Quantity, Price, Remove)

Order Summary: Subtotal, Shipping, Tax, Total

"Continue Shopping" and "Proceed to Checkout" buttons

Checkout Page

Shipping Address Form

Order Review (Items, Prices, Totals)

"Place Order" button

Terms and Conditions note

Admin Dashboard

Sidebar Navigation: Dashboard, Products, Orders, Users

Product Management: Add New button, Product list with Edit/Delete

Order Management: Order list with status filters

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Bottom Navigation

[🏠 Home] [🔍 Search] [🛒 Cart] [👤 Profile]

Home Screen

Search Bar

Product Grid (2 columns)

Swipe gestures for quick actions

Pull to refresh

Product Detail Screen

Back arrow

Product image (swipeable gallery)

Product info

Quantity selector

"Add to Cart" fixed bottom button

Cart Screen

Edit mode for quantity updates

Swipe to remove items

Order summary sticky bottom

Checkout button

Checkout Flow

Step indicator: Cart → Shipping → Payment → Confirm

Address form (auto-complete)

Order summary

Place order button

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#2563EB), Secondary (#7C3AED), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript

Day 2: Authentication pages (login, register)

Day 3: Product listing page

Day 4: Product detail page

Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements